

■ Discord Listings Bot

Complete AI Prompt — Replit Build Guide

This document contains the full prompt to send to Replit AI to build the Listings Panel system. Paste the entire contents of each section into Replit AI alongside your existing bot code.

■■ Do NOT replace anything already in the bot. Only add new features as described.

1. Overview & Instructions

Add a full Listings Panel system to my existing Discord.js bot. Do NOT replace anything already in the bot — only add new features. Use Discord embeds, buttons, modals, and select menus. Use JSON or SQLite (whichever fits the existing bot) for persistence.

2. Setup Command — /listingsetup (Admin Only)

Run once to initialize the system:

- Bot posts a permanent panel embed in the current channel
- This panel NEVER gets deleted — it is the entry point for all users
- Store the panel message ID so it can be edited/refreshed, not reposted

Main Panel Embed

```
■ ■ Server Marketplace ■  
■ ■  
■ Buy, sell, bid, and trade items ■  
■ with other players safely through ■  
■ the bot. ■  
■ ■  
■ Click below to get started. ■
```

Buttons:

[\[■ Create Listing\]](#) [\[■ Browse Listings\]](#) [\[■ My Listings\]](#)

■ Preview auto-expires after 120 seconds if no action is taken. Delete it and require the user to start over.

4. Live Listing Embed (Posted to #listings Channel)

[illegible]

Buttons:

☐ Buy Now ☐ Make Offer ☒ Place Bid

[■ Watch] [■ Cancel Listing*]

*Cancel only functional for the lister and admins

5. Button Behaviors

[■ Buy Now]

- Ephemeral confirm prompt: "Confirm purchase of [Item] x[Qty] for \$[Price]?" with [■ Yes, Buy] and [■ No]
- On confirm: Create private ticket channel #txn-auc4821-buyer
- Add buyer and seller to the channel
- Post pinned transaction summary embed with item, price, buyer, and seller
- Immediately after summary, bot posts the Transaction Panel (see Section 7)
- DM seller: "■ [Item] was purchased by @Buyer for \$X"
- Mark listing [SOLD], disable all buttons, update embed status

[■ Make Offer]

- Opens Modal with: Offer Amount and Message to seller (optional)
- Bot DMs seller with offer embed including [■ Accept], [■ Decline], [■ Counter Offer] buttons
- Accept → create ticket channel, post summary + Transaction Panel
- Decline → bot DMs buyer: "Your offer was declined"
- Counter → modal asks seller for counter amount → DM sent to buyer with [■ Accept Counter] [■ Decline]
- Accept Counter → create ticket, post summary + Transaction Panel

[■ Place Bid]

- Check if user has set their IGN (/setign). If not: ephemeral "Set your IGN first with /setign [name]"
- Fetch balance from Donut SMP API: GET <https://donutsmp.net/api/v1/balance/{ign}> (configurable in config.json)
- If balance < current bid + min increment: Ephemeral "■ Insufficient balance. You need at least \$X"
- If sufficient → Modal: "Enter your bid (minimum \$X)"
- On valid bid: Update listing embed (current bid, bidder tag)
- DM previous highest bidder: "■ You were outbid on [Item]"
- DM seller: "■ New bid of \$X by @User on [Item]"
- If bid placed within last 3 min of end → extend timer by 3 min (anti-snipe) and update embed: "■ Extended — anti-snipe triggered"

[■ Watch]

- Adds user to listing's watcher list
- Ephemeral: "■ You'll be notified of updates on this listing"
- DM all watchers on: new bid, offer accepted, and listing sold/expired

[■ Cancel Listing]

- Only works for the lister or users with the admin role
- Confirm prompt: "Are you sure? This will remove the listing."
- On confirm: mark [CANCELLED], disable all buttons, DM all watchers that listing was cancelled

6. Ticket Channel — Transaction Summary

When a ticket is created from any path (Buy Now, accepted offer, or won bid):

- Create private channel: #txn-[listingid]-[buyername]
- Add buyer and seller
- Post and PIN a transaction summary embed (item, price, buyer, seller)
- Immediately post the Transaction Panel embed below it (also pinned)

7. Transaction Panel (Deal Flow)

[illegible]

Buttons:

[■ Deal Done] [■ Scam Vouch]

Deal Done — First Party Clicks [■ Deal Done]

- Bot edits the panel embed to show waiting state and pings the other party
- [■ Deal Done] button is disabled and relabeled: "■ Confirmed by @[FirstParty]"
- Other party's button becomes active, labeled: "■ Confirm Deal Complete"

Deal Done — Second Party Confirms

- Call existing vouch system: addVouch(userId) for BOTH parties — do not rebuild the vouch system
- Update the Transaction Panel embed to show completion with both parties' names and +1 vouch each
- Disable all buttons on the panel
- DM both parties confirming the vouch was awarded
- Wait 5 minutes then archive/delete ticket using the existing ticket close method
- Log to admin log channel: Item, Buyer, Seller, Final Price, Type, Timestamp, Vouch outcome

Completed embed:

```

■ ■ Deal Complete ■
■ ■
■ Both parties have confirmed. ■
■ Vouches have been awarded to: ■
■ • @Buyer (+1 vouch) ■
■ • @Seller (+1 vouch) ■
■ ■
■ This ticket will close in 5 min. ■
```



8. Scam Vouch Flow

A Party Clicks [■ Scam Vouch]

Bot sends ephemeral warning to the clicker asking them to confirm. Warning states this is serious and false claims can be appealed. Buttons: [■ Yes, Submit Scam Vouch] and [■ Cancel]

On Confirmed Scam Vouch

- Call existing vouch system: `addScamVouch(userId)` — do NOT rebuild it
- Update Transaction Panel embed to show scam vouch filed, with [■ Appeal Scam Vouch] button (only clickable by the accused; greyed out for reporter)
- DM the accused: scam vouch filed, can appeal via button in ticket
- DM the reporter: scam vouch submitted and logged
- Log to admin log channel: Reporter, Accused, Item, Ticket, Timestamp, Status: Scam Vouch Filed
- Lock the ticket channel — neither party can send messages, but both can still read it

Scam Vouch embed:

```

■ ■ Scam Vouch Submitted ■
■ ■
■ @[Reporter] has filed a scam vouch ■
■ against @[Accused]. ■
■ ■
■ @[Accused]: If this is false, you ■
■ can appeal using the button below. ■
■ A support ticket will be opened. ■

```

[■ Appeal Scam Vouch*] *Accused only

9. Appeal Flow

Accused Clicks [■ Appeal Scam Vouch]

Opens a Modal with: Reason for appeal (long text, required) and Evidence (optional — image links or description of screenshots).

On submit:

- Create new private support ticket channel: #appeal-[listing-id]-[accused-username]
- Add accused party and all users with admin role
- Post appeal embed with appellant, reporter, listing reference, reason, evidence, and timestamp
- Show admin-only buttons: [■ Appeal Approved] and [■ Appeal Denied]
- Send ephemeral to accused confirming appeal submitted
- Ping admin role in the appeal channel

Admin Clicks [■ Appeal Approved]

- Bot prompts admin for confirmation (ephemeral)

- On confirm: call removeScamVouch(userId) from existing vouch system
- DM accused: appeal approved, scam vouch removed
- DM reporter: scam vouch was reviewed and removed by staff
- Log to admin log: Appeal Approved, vouch removed, admin who approved, timestamp
- Close appeal ticket after 5 minutes

Admin Clicks [■ Appeal Denied]

- Bot prompts admin for a denial reason via modal
- DM accused: appeal denied, reason given, scam vouch remains
- Log to admin log: Appeal Denied, reason, admin who denied, timestamp
- Close appeal ticket after 5 minutes

10. Vouch System Integration Notes

■■ DO NOT build a new vouch system. The existing vouch system is already in place. Only integrate with it.

Find the existing functions that handle vouching and call them directly:

```
addVouch(userId)           // positive vouch
addScamVouch(userId)       // scam/negative vouch
removeScamVouch(userId)    // remove scam vouch
```

If these are slash command handlers, extract the core logic into reusable utility functions so they can be called from button interactions without invoking a slash command. All vouch counts and seller ratings shown in listing embeds must pull live from the existing vouch system's data store — not a separate store.

11. Commands

/setign [ign]

- Saves Minecraft IGN linked to the user's Discord ID
- Optional: validate the IGN against the Mojang API
- Confirm: "■ IGN [name] registered to your account."

/mylistings

- Ephemeral panel showing all the user's active listings
- Shows status, current bid if bidding, time remaining
- Each listing has: [View] [Cancel] [Relist*] buttons
- *Relist only on expired/unsold — reposts without re-entering all info

/listinghistory

- Paginated list of completed listings
- Users see own history; admins see all
- Shows final price, buyer/seller, date, outcome

/listingadmin (Admin Only)

```
/listingadmin setlistingchannel #channel
→ Sets where listing panels are posted

/listingadmin setticketcategory [category id]
→ Sets which category ticket channels are created under

/listingadmin setlogchannel #channel
→ Sets where all auction/deal events are logged

/listingadmin setappealcategory [category id]
→ Sets which category appeal ticket channels go under

/listingadmin forceclose [listing_id]
→ Forcefully cancels an active listing
```

/listingadmin refreshpanel

→ Re-posts the main marketplace panel if buried or deleted

12. Bid Expiry System

Use node-cron to check every 30 seconds for expired bidding listings.

On Expiry WITH Bids

- If reserve price was set and not met: update embed to [RESERVE NOT MET], DM seller and all watchers
- If reserve met or no reserve: open ticket between winner and seller
- Post transaction summary + Transaction Panel in the ticket
- DM both parties with results
- Update listing embed to [■ SOLD — AUCTION ENDED]
- Log to admin log channel

On Expiry WITH NO Bids

- Update embed to [■ EXPIRED — NO BIDS]
- DM seller
- DM all watchers

13. Additional Features

- **Listing ID** — every listing gets a unique short ID (e.g. AUC-4821) shown in the embed footer
- **Pixel-Perfect Preview** — the preview before posting is exactly what appears live in the listings channel
- **Edit Window** — seller can edit description or price within 5 minutes of posting if no bids or offers placed yet. Edit button appears in /mylistings for recent listings only
- **Anti-Snipe** — bids placed within the final 3 minutes of a bidding listing extend the timer by 3 minutes. Embed updates to show: "■ Extended — anti-snipe triggered"
- **Category Browse** — [■ Browse Listings] on the main panel opens a select menu to filter active listings by category (Spawners / Bases / Items)
- **Watchlist** — [■ Watch] on any listing adds the user to that listing's watcher list. DM on new bids, offer accepted, sold/expired
- **Relist** — expired or unsold listings in /mylistings have a [Relist] button to instantly repost without re-entering all info
- **Feedback System** — after ticket closes following confirmed deal, both parties get a DM to rate each other 1-5 stars. Ratings stored and displayed on future listings as "Seller Rating: ■■■■"
- **Sold Listings Log** — every transaction (item, final price, buyer, seller, type, timestamp) logged to admin log channel
- **Preview Auto-Expiry** — if seller takes no action on preview after 120 seconds it is automatically deleted and they must start over

14. Log Channel Events

All of the following events are posted to the configured log channel:

- Deal Confirmed
→ Item, Buyer, Seller, Price, Both vouched, Timestamp
- Listing Posted
→ Item, Seller, Category, Type, Listing ID, Timestamp
- Listing Cancelled
→ Item, Seller, Listing ID, Cancelled by, Timestamp
- Buy Now Executed
→ Item, Buyer, Seller, Price, Listing ID, Timestamp
- Bid Placed
→ Item, Bidder, Amount, Previous Bidder, Listing ID, Timestamp
- Listing Expired
→ Item, Seller, Listing ID, Outcome, Timestamp
- Scam Vouch Filed
→ Reporter, Accused, Item, Ticket, Timestamp
- Scam Vouch Appealed
→ Accused, Reporter, Appeal channel, Timestamp
- Appeal Approved
→ Accused, Admin, Vouch removed, Timestamp
- Appeal Denied
→ Accused, Admin, Reason, Timestamp

15. Database Schema

Listings table:

```
id, seller_id, item_name, description, quantity,  
category (spawners / bases / items),  
type (auction / bidding),  
buy_now_price, starting_bid, current_bid,  
current_bidder, min_increment, reserve_price,  
duration_ms, created_at, ends_at,  
status (active / sold / expired / cancelled),  
message_id, channel_id,  
bid_history [ { user_id, amount, timestamp } ],  
watchers [ user_ids ]
```

Users table:

```
discord_id, ign,  
watchlist [ listing_ids ]
```

Note: Vouch and rating data lives in the existing vouch

system. Do not duplicate it here.

16. Config File (config.json additions)

```
{
  "listingChannelId": "",
  "listingPanelMessageId": "",
  "ticketCategoryId": "",
  "appealCategoryId": "",
  "logChannelId": "",
  "balanceApiUrl": "https://donutsmp.net/api/v1/balance/",
  "adminRoleId": "",
  "previewTimeoutSeconds": 120,
  "ticketCloseDelayMs": 300000
}
```

17. Technical Requirements

- All interactions handled via interactionCreate
- All modals use unique customIds prefixed with "listing_"
- All collectors have proper timeout cleanup to prevent memory leaks
- All API calls and database writes wrapped in try/catch
- Preview embeds auto-delete after 120 seconds if no action is taken
- The existing ticket close method is used for closing both transaction tickets and appeal tickets — do not build a new one
- Vouch functions are extracted into reusable utilities — do not rebuild the vouch system
- Balance API URL is configurable in config.json — do not hardcode it